

ECSE 316 Assignment 2 Report

1. Introduction

Fourier techniques are essential tools in signal and image analysis because they break down complex data into its underlying frequency components. This report presents the development and evaluation of both the straightforward Discrete Fourier Transform (DFT) and the more efficient Fast Fourier Transform (FFT). Through experiments involving denoising, compression, visualization, and runtime comparison, we highlight how these transforms behave in practice and examine their computational performance.

2. Design

2.1. Overview

This project is centered around 4 modes, each mode focuses on a different utilization of the fourier transform.

1. Mode 1: FFT Visualization
2. Mode 2: Image Denoising
3. Mode 3: Image Compression
4. Mode 4: Runtime Complexity Analysis

2.2. Naive Discrete Fourier Transform (DFT)

2.2.1. 1D DFT

The naive_dft function implementation directly follows the mathematical definition:

$$X_k = \sum_{n=0}^{N-1} x_n \cdot e^{-2\pi i \frac{kn}{N}} \quad \text{for } k = 0, 1, \dots, N-1$$

We utilized a nested loop structure, where the outer loop iterates through each frequency component, and the inner loop computes the summation of the input samples weighted by the complex exponentials. The computational complexity is $O(N^2)$ as there are N outputs and each requires summing N inputs.

2.2.2. 2D DFT

To extend the one dimensional DFT to its two dimensional form, the transform is applied first to each row and then to each column of the image matrix. The computational complexity is $O(N^3)$ for an $N \times N$ matrix.

2.3. Fast Fourier Transform (FFT)

2.3.1. 1D FFT

The algorithm utilizes the Cooley-Turkey FFT technique, which greatly reduces computational cost by recursively splitting the DFT into smaller DFTs of even and odd indices. This recursion structure lowers the computational complexity from $O(N^2)$ to $O(N \log N)$.

2.3.2. 2D FFT

To extend the one dimensional FFT to its two dimensional form, the transform is applied first to each row and then to each column of the image matrix. The computational complexity is $O(N^2 \log N)$ which is significantly faster than the Naive 2D DFT.

2.4. Inverse FFT (IFFT)

Computes the inverse Fourier Transform by taking the complex conjugate of the input, passing it through the existing forward `cooley_tukey_fft`, and then taking the conjugate of the result scaled by $1/N$. The time complexity is the same as that of the forward FFT.

2.5. Handling Arbitrary Image Sizes (Padding)

The Cooley-Tukey algorithm requires the input length N to be a power of 2. Since actual images might not satisfy this condition, the `pad_to_shape` function is used to zero pad the image to the next power of two. This avoids index mistakes and guarantees a balanced recursion tree.

2.6. Application Algorithm

2.6.1. Denoising (Low-Pass Filter)

The denoising algorithm pads the image to a power-of-two size and computes its 2D FFT. A circular low-pass filter is applied in the frequency domain using `fftshift` to identify the spectrum center, removing high frequency noise while preserving low frequency structures. The inverse FFT is then performed, and the result is cropped and clipped to produce the denoised image.

2.6.2. Compression (Magnitude Thresholding)

The compression method calculates the image's 2D FFT to get the image's frequency coefficients. Only the largest-magnitude coefficients are retained for each compression level and the remaining coefficients are set to zero. After that, it uses the retained coefficients to recreate the image using the inverse FFT. More coefficients are eliminated at higher compression levels, which reduces detail while preserving data.

3. Testing

3.1. Application Testing

To validate the accuracy and reliability of our system, we carried out a broad set of functional tests for every mode:

3.1.1. Mode 1 (FFT Visualization)

Checked that the displayed frequency spectrum correctly represents the original image by comparing our output to the result produced by NumPy.

3.1.2. Mode 2 (Image Denoising)

Evaluated the denoising pipeline by applying low pass filters with different cutoff values and confirming that noise was reduced without removing important visual information.

3.1.3. Mode 3 (Image Compression)

Assessed the compression workflow by removing various proportions of Fourier coefficients and examining the resulting impact on image clarity and file size.

3.1.4. Mode 4 (Runtime Analysis)

Verified that the recorded execution times for the naive DFT and FFT implementations matched expected performance trends and theoretical behavior.

3.2. Edge Case Testing

3.2.1. Non Power of Two Size

Ensured that the padding and resizing logic correctly processed images whose dimensions are not powers of two, which is required before applying FFT based operations.

3.2.2. Very High Compression Levels

Ran tests at the lowest and highest compression settings (0 percent and 99.9 percent) to observe how the system behaves when keeping nearly all or almost none of the frequency components.

3.3. Performance Testing

To test performance, we wanted to determine how well the naive DFT and FFT algorithms scale as the problem size increases. We measured and compared execution times for both algorithms over a sequence of randomly generated matrices with sizes 32 x 32, 64 x 64, 128 x 128, and 256 x 256.

4. Analysis

4.1. Runtime of the Naive DFT for 1D

The naive 1D Discrete Fourier Transform (DFT) is defined as:

$$X_k = \sum_{n=0}^{N-1} x_n \cdot e^{-2\pi i \frac{kn}{N}} \quad \text{for } k = 0, 1, \dots, N-1$$

To compute the DFT, the algorithm must produce N output frequency components, and each component requires summing over N input samples..

Per Output Component: $O(N)$

Total Time complexity: $O(N^2)$

4.2. Runtime of the FFT for 1D

The Fast Fourier Transform (FFT) uses a divide and conquer approach, implemented by the Cooley-Turkey algorithm. The algorithm splits the input into its even indexed and odd indexed samples, creating two FFT subproblems of size $N/2$. It recursively computes FFT for both halves, and then combines the result using $O(N)$ additional work per level. This combination, called a butterfly operation, involves one complex multiplication and two additions for each pair of elements.

This gives the following recurrence relation: $T(N) = 2T(N/2) + O(N)$

Here: $a = 2$, $b = 2$ and $f(N) = O(N)$.

Solving the recurrence via Master Theorem:

Master Theorem states for a recurrence: $T(N) = aT(N/b) + f(N)$

By computing $N^{\log_b a}$ we get: $\log_2 2 = 1 \Rightarrow N^{\log_2 2} = N$

By comparing $f(N)$ with $N^{\log_b a}$: $f(N) = O(N) = \Theta(N^{\log_2 2}) = \Theta(N)$

By Master Theorem Case 2, the solution to the recurrence is: $T(N) = \Theta(N \log N)$

Time Complexity: $O(N \log N)$

4.3. 2D Complexity Analysis

A 2D transform is implemented by first applying the 1D transform to each row, and then applying the 1D transform to each column of the resulting matrix.

4.3.1. Naive 2D DFT

Row Transformation:

- Cost per Row: $O(N^2)$
- Total Row Cost = $N \times O(N^2) = O(N^3)$

Column Transformation:

- Cost per Column: $O(N^2)$
- Total Column Cost = $N \times O(N^2) = O(N^3)$

Total: $O(N^3) + O(N^3) = O(N^3)$

4.3.2. FFT 2D

Row Transformation:

- Cost per Row: $O(N \log N)$
- Total Row Cost = $N \times O(N \log N) = O(N^2 \log N)$

Column Transformation:

- Cost per Column: $O(N \log N)$
- Total Column Cost = $N \times O(N \log N) = O(N^2 \log N)$

$$\text{Total: } O(N^2 \log N) + O(N^2 \log N) = O(N^2 \log N)$$

4.3.3. Comparison

When comparing the two algorithms, the computationally expensive $O(n^3)$ time complexity of the naive 2D DFT causes it to scale poorly with increasing input size. By lowering the runtime to $O(N^2 \log N)$, the 2D FFT, on the other hand, provides a notable improvement and is far more practical and efficient for handling large N.

5. Experiment

5.1. FFT Visualization

Goal: We wanted to compare the FFT magnitude spectrum that was generated by the FFT2D implementation with the numpy implementation to check the correctness

Methodology:

1. Loading the Image: we converted to image to greyscale and then if it was required we padded it to the nearest power of 2 dimensions
2. Compute FFT: we applied the FFT2D function to compute the 2D FFT
3. Computed Magnitude spectrum: For both FFT results we calculated the logarithm of the magnitude to enhance the visibility
4. Plotting: For visual comparison we showed the original image beside both FFT magnitude spectra

Result:

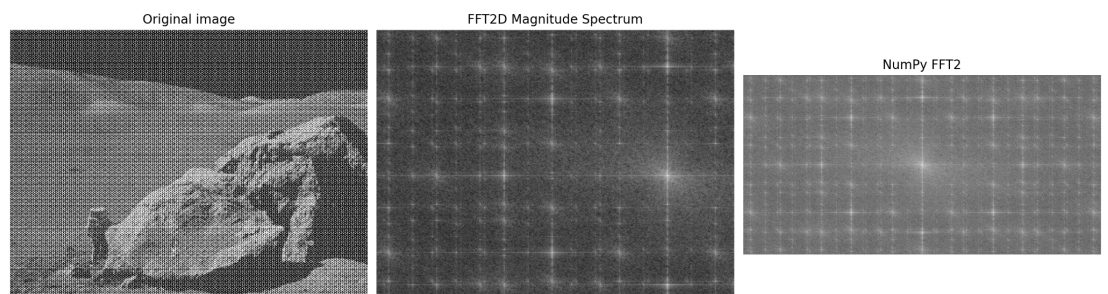


Figure 1: Comparison of the FFT magnitude spectra computed using FFT2D and NumPy's FFT2.

Discussion:

The alignment of the magnitude spectra produced by our methods demonstrates the correctness of the FFT implementation, as both results show a visually consistent representation of the frequency content. Small differences may appear due to computational precision or minor implementation details, but overall, the spectra are essentially identical, confirming that our FFT produces accurate results.

5.2. Image Denoising

Goal: Use the FFT to denoise the image by filtering out high-frequency components, which will reduce noise while keeping important details intact.

Methodology:

1. Loading the Image: Ran the 2D FFT on the original input image.
2. Apply Low pass filter: Set all Fourier coefficients outside a chosen cutoff radius to zero in the frequency domain.
3. Inverse FFT: Applied the inverse FFT to reconstruct the filtered image.
4. Repeat: Tested several cutoff radii to observe different levels of denoising.

Result:

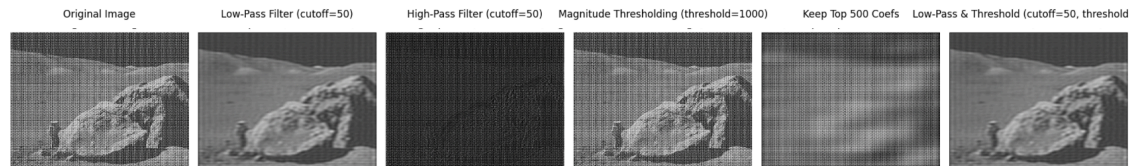


Figure 2: Comparison of denoised images across various parameters

Discussion:

As the cutoff radius increases, more high-frequency information is preserved, which maintains finer textures but also allows additional noise to remain. Removing low frequencies caused a noticeable drop in noise, although this came at the cost of losing some subtle image features. In contrast, removing high frequencies was more effective for denoising: it significantly reduced noise while keeping the main structure and essential details intact, even though some small scale textures were inevitably removed.

The thresholding experiments showed that a cutoff radius of 30 offered the most balanced result. It removed the majority of high-frequency noise while still preserving important visual features. Increasing the cutoff to 50 kept more detail but also retained more noise, while raising it to 70 produced almost no further improvement, since most noise had already been eliminated at lower thresholds. At very high cutoffs, the image even lost a bit of sharpness without any real benefit. Overall, a moderate high-frequency cutoff around 30 provided the best compromise between denoising and detail preservation.

5.3. Image Compression

Goal: Compress the image size by setting a chosen percentage of Fourier coefficients to zero and evaluate how this affects file size and visual quality.

Methodology:

1. Set Compression Levels: Used the following percentages for compression: [0%, 50%, 90%, 95%, 99%, 99.9%].
2. Apply Compression to FFT: For each level, preserve the top (100 - X)% highest-magnitude Fourier coefficients and remove the remaining ones.
3. Perform Inverse FFT: Reconstructed the image from its compressed Fourier representation using the inverse FFT.
4. Store Sparse Matrices: Saved the compressed Fourier arrays to disk and recorded their resulting file sizes.
5. Visualization: Displayed the original and each compressed reconstruction using a 2×3 layout to compare quality across levels.

Result:

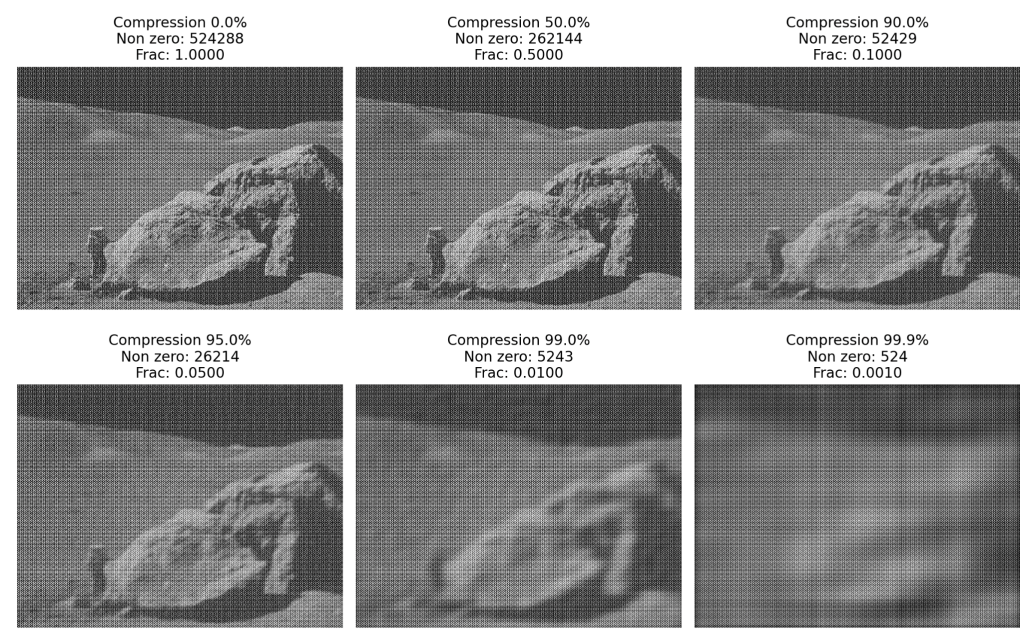


Figure 3: Compressed images across different levels

Table 1: Sparse Matrix Sizes at Various Compression Levels

Compression Level (%)	File Size (KB)	Reconstruction Quality
0	7834	Original Image
50	4082	Slightly compressed, details mostly preserved
90	820	Noticeably compressed, some fine details lost
95	405	Strong compression, image appears blurred

99	75	Very aggressive compression, detail largely lost
99.9	7.48	Extreme compression, image barely recognizable

Discussion:

As compression increased, the size of the saved sparse Fourier matrices consistently decreased because more coefficients were set to zero. At low compression levels, the reconstructed images remained very close to the original and retained nearly all of their visual clarity. As the compression became more extreme, especially beyond approximately 95%, the reconstructions began to show noticeable artifacts such as blurring, loss of fine detail, and distortions in the image structure. From all the compression levels tested, a compression level of about 90% provided the best overall balance. It achieved a substantial reduction in file size while still producing a reconstruction that remained clear and recognizable, offering a strong compromise between memory efficiency and visual quality.

5.4. Runtime Analysis

Goal: Evaluate how the runtimes of the naive DFT2D and the optimized FFT2D scale as the input size increases.

Methodology:

1. Choose Input Sizes - Used matrices with dimensions equal to powers of two: [8, 16, 32, 64, 128, 256].
2. Create Random Test Data - For each dimension, generated a random $N \times N$ complex valued matrix.
3. Measure Execution Time - For each matrix, executed both DFT2D and FFT2D ten times and logged each runtime measurement.
4. Analyze Results - Computed the mean and standard deviation of the recorded runtimes for both algorithms at all tested sizes.
5. Plotting - Graphed the performance data to compare growth rates and visualize how algorithmic complexity changes with problem size.

Result:

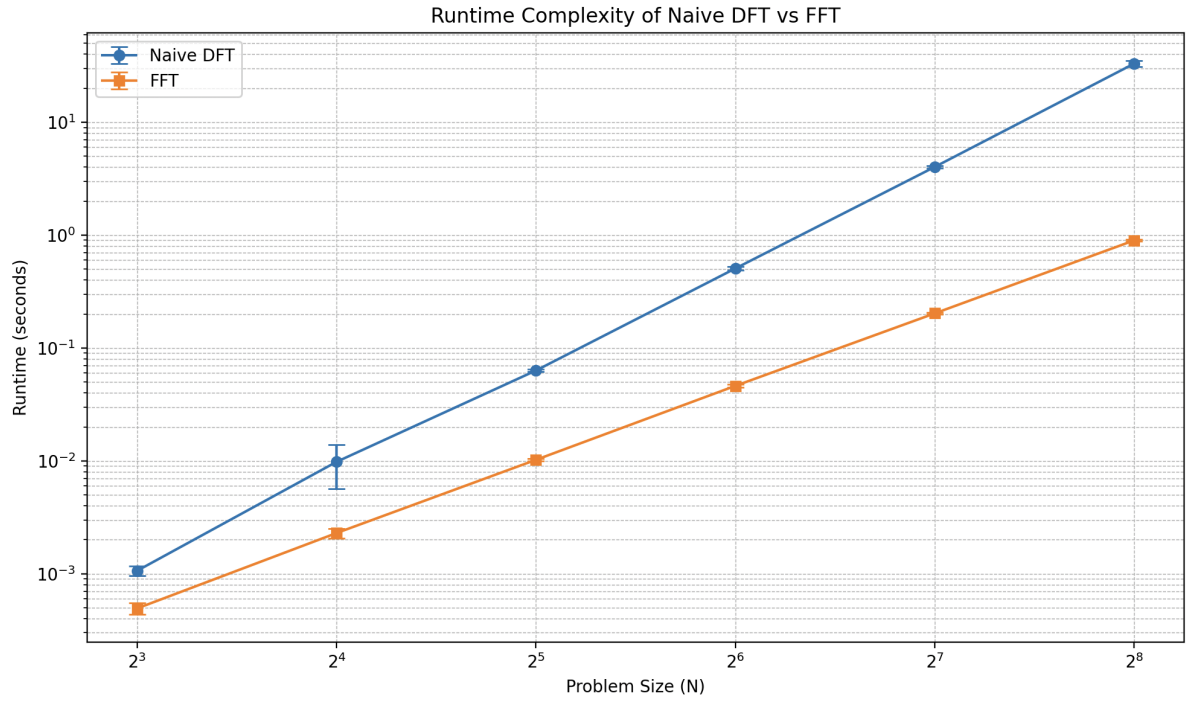


Figure 4: Computational Time of Naive DFT vs FFT

Table 2: Timing Statistics for Naive DFT vs FFT

Problem Size (N)	Naive DFT Mean (s)	Naive DFT Std Dev (s)	FFT Mean (s)	FFT Std Dev (s)
8	0.0011	0.0001	0.0005	0.0000
16	0.0098	0.0021	0.0023	0.0001
32	0.0632	0.0007	0.0102	0.0001
64	0.5069	0.0090	0.0462	0.0007
128	4.0229	0.0545	0.2027	0.0013

256	33.1120	1.1119	0.8987	0.0103
-----	---------	--------	--------	--------

Discussion:

The runtime measurements show a clear difference in how the two algorithms scale as the matrix size increases. The naive two dimensional DFT exhibits a rapid rise in runtime, growing quadratically and becoming impractical for larger inputs. In contrast, the FFT-based method scales much more slowly and maintains near logarithmic growth, which allows it to handle significantly larger matrices within reasonable time. At every tested problem size, the FFT completed its computations far more quickly than the naive approach, and this performance gap widened dramatically as the dimensions increased. The repeated trials produced very small standard deviations, which indicates that both methods run consistently and that the averaged timings accurately reflect their true performance.

6. Conclusion

In this assignment, we successfully implemented and analyzed both naive and optimized algorithms for the discrete Fourier transform, including 1D and 2D DFTs, the Cooley–Tukey FFT, and their inverses. We verified correctness, efficiency, and robustness, including edge cases, through systematic testing on images and synthetic signals. The significant computational reductions attained by the Cooley–Tukey algorithm, especially for large scale 2D transforms, were clearly demonstrated by the comparison between naive DFT and FFT. The main challenges encountered during this work included handling image sizes that were not powers of two, ensuring numerical stability for very small or very large coefficients, and designing an effective compression and denoising scheme that balances visual quality with computational efficiency. Additionally, optimizing the recursion threshold for the FFT to avoid overhead at small sub-problems required careful consideration.

Possible improvements could involve using an iterative version of the FFT to cut down on recursion overhead, trying hybrid compression strategies that keep the important low frequencies while selectively retaining high frequency details, or parallelizing the FFT for handling larger images or real-time processing. We could also experiment with adaptive cutoff thresholds for denoising or explore other FFT variants, like radix-4 or mixed-radix FFTs, to make the algorithm more efficient for specific problem sizes.

Overall, this work demonstrates the practical utility of Fourier methods in signal and image processing, highlights the importance of careful algorithm design and testing, and illustrates how performance assessment can guide meaningful optimizations in computational methods.

7. References

NumPy, “Overview — NumPy v1.19 Manual,” *numpy.org*, 2022.
<https://numpy.org/doc/stable/>

Matplotlib, “Overview — Matplotlib 3.4.1 documentation,” *matplotlib.org*.
<https://matplotlib.org/stable/contents.html>